

Giovanni's blog

Sunday, July 26, 2026

RedHat 7.2 (2001), Abiword 1.0, & Sanger's Rule towards kernel development

Search This Blog

Search

[Home](#)

[Report Abuse](#)

[Blog Archive](#)

[July 2026](#) (4)

[June 2026](#) (7)

[May 2026](#) (1)

[April 2026](#) (4)

[January 2026](#) (2)

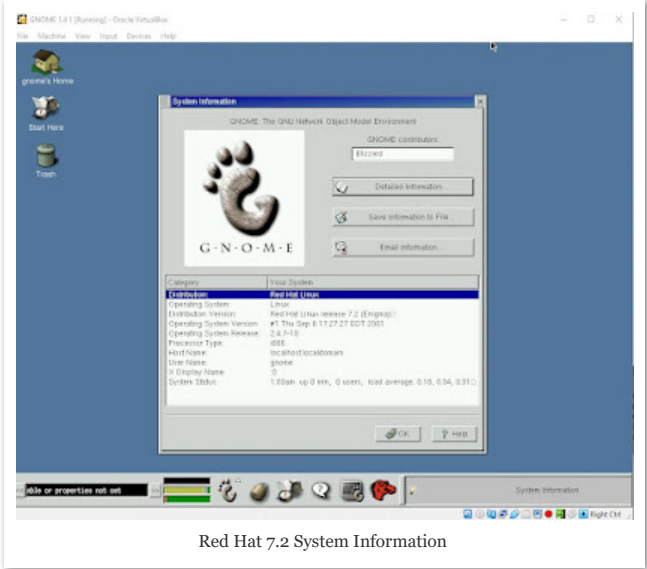
[December 2025](#) (3)

[September 2025](#) (1)

[August 2025](#) (1)



Red Hat 7.2 (2001) with 8MB RAM



Red Hat 7.2 System Information

Two nights ago (technically Saturday night) I spent some time using free tokens from Replit, Github Codespace, and Gemini to understand for myself what goes into kernel and system development. While I have read [Linux From Scratch](#) excerpts and countless resources, sometimes it is more interesting (or at a loss of ideas?) to see AI attempt to develop an operating system image. The first few attempts I got a non bootable image, around 5-6MB in size. The later ones determined what was missing and I ended up with

a stalled bootup process (further than before) at around 20-29MB (the request was to make it less than 30MB, which it did). Those sizes were requested because I knew they were possible and bootable (such as [Tiny Core Linux](#) and [Slitaz](#)'s core ISO- their images are also bootable in under 30MB). However, without knowing enough steps, the code assistant can sometimes repeat the same mistake a few times, and it wastes tokens. An experienced kernel developer doesn't need any code assist, and one who is somewhat familiar could probably spot a few common errors. I have neither expert nor intermediate experience, therefore my attempt was as good as fresh. This might be some people's cue to say, "*read up on a human written tutorial!*" and such, but if you've known I've had this interest in developing small OSes since 2021, you might wonder why I am trying this now. I will not try to persuade you nor vice versa. AI has been more useful as a fetcher and process instructor (even when it gets things wrong, I still learn a thing or two).

For example, while I got CodeSpaces to generate a few ISOs, the next day I pivoted to searching the components that AbiWord used- initially I thought it could use libc5, but realized it used [glibc 1.2](#). I realized I was trying to build an OS with a different library when I thought it was the only lightweight one. However, I wanted to know how much RAM it used, and wondered whether it would be simpler to install and boot RedHat 7.2, which was released in 2001, to load AbiWord with [GNOME 1.4](#). According to a system requirements page from an [archived MIT page](#), it doesn't technically need GNOME 1.4, and I could [run it with a lighter xfce or even Fluxbox](#). Sure, I've used both of those before, but how much time will it take me to compile a distro from 2001 with xfce or fluxbox? So I decided to test RedHat 7.2. A couple years ago, I did download a few Mandrake and RedHat distros from that era, but I had a super difficult time getting past the installation screen. And today I found a VirtualBox appliance that allowed me to run RedHat 7.2 like a LiveCD- it was hosted on a [GNOME page](#), which appeared to be an archive around the time that their GNOME 3 launched in 2012.

On a super basic level, the few tests I really wanted to know was what was the absolute minimum amount of RAM that RedHat 7.2 really needed, even if I wasn't planning on thinning it down from a full starting point- rather, I planned to develop a kernel from scratch with Abiword preinstalled, since I could figure out what I didn't need as I removed things iteratively. However, I do realize the two ISOs- from Slitaz or TinyCore which feature GTK, and the X server based RedHat, are *far* different, both in terms of kernel version (the former use kernels 5.0 and 6.x, whereas RedHat 7.2 used 2.4, and there would be quite a bit apples to oranges to spot. That said, if I could get RedHat to run on 16MB or even 12MB or even MB, that would bode quite well on paring it down to 8MB. The system requirements were 32MB in 2001:

This appendix provides you with information on the contents of the CD-ROM that accompanies this book.

Two components are included on the CD:

- Red Hat Linux 7.2
- The Red Hat Linux 7.2 Weekend Crash Course Assessment Test

Also included is an electronic, searchable version of the book that can be viewed with Adobe Acrobat Reader.

System Requirements

Make sure that your computer meets the minimum system requirements listed in this section. If your computer doesn't match up to most of these requirements, you may have a problem using the contents of the CD.

- PC with a Pentium processor running at 90 Mhz or faster
- At least 32MB of RAM, 64MB or more if you will use the X Window system
- Between 300MB and 2.4GB of free disk space, depending on installation options (1.2GB or more recommended)
- Ethernet network interface card (NIC) or modem with a speed of at least 28,800 bps
- A CD-ROM drive — double-speed (2x) or faster
- A 3.5" floppy disk drive for creating a boot floppy (optional)

You will need at least 2.4GB of hard drive space to install all the software from this CD.

What's on the CD

The CD-ROM contains source-code examples, applications, and an electronic version of the book. Following is a summary of the contents of the CD-ROM arranged by category.

"From *Red Hat Linux 7.2 Weekend Crash Course*" (ISBN: 0764536427)

Remember when I mentioned AbiWord doesn't need GNOME 1.4? Well, I reasoned that a lot of the extra software preinstalled in RedHat most likely contributes to the minimum RAM requirements, and I ran several tests today in Virtual Box, from 64MB to 7MB. 7MB could reach the bootloader, but it could not proceed further than that. 16MB could load most applications, and even 12MB, albeit a bit slower. It would also lag if multiple windows like Nautilus were open while a browser and/or other smaller apps were open.

RedHat 7.2 (GNOME 1.4) with 12MB RAM and 1MB VRAM

Thus I can see how reasonable it is to require something like 32MB RAM. That lag wouldn't happen if one is multitasking. So using this logic, I realized (many years ago), that I could optimize a RedHat-like distro to run just Fluxbox or FLTK/FLWM, or xfce (which was released in 1997) to run Abiword as a single application. Gnumeric could load with just 10MB of RAM, which is also a lightweight spreadsheet. That said, Abiword itself uses **less than 2MB** of RAM. Therefore X server is the largest component of the memory usage when 8 or 12MB of RAM is loaded. Therefore if I can reduce the size of the ISO to 8MB, then I could potentially lower the power consumption of the entire system by 10-20% (and far more if one compares 2000-era DRAM to HBM3e RAM (600nm > 2nm GAA). Wondering how I am getting to this figure? This [article](#), and my [January article](#), "The NM10 Chipset, Backporting to the Pentium Era, and Makimoto's Wave" will fill you in. It's all about the energy consumption of a 1mm²-4mm² 18NA chip produced using EUV by one of ASML's latest lithography machines. The idea is if you can produce a small enough chip with enough embedded DRAM, you could sell enough chips per wafer- 52,000 or so at a price such as \$2-4, and therefore competitive with Apple or Nvidia which reserves the same \$45,000 2nm wafers for large GB300 chips and where yields might be 89 per 300mm wafer, thus sold for datacenters at hundreds of thousands of dollars each to recoup the cost. Then **each OS could run as a single app** as a multi boot from a microSD/USB, or wherever the HDD/SSD stores the liveISO. Thus instead of having one 400MB ISO with tens of apps one live ISO, tens or *hundreds* of RedHat 7.2 ISOs would be stored on the microSD card at just 20-30MB in size each, one for each app, such as Gnumeric and Abiword. How would one person develop that?

Sanger's rule applied to AI objectively suggests... a discovery may be possible in absence of the funding for three employees who could do the same.

"Sanger's rule ... anytime you get technical development that's two to threefold or more efficient, accurate, cheaper, a whole range of experiments opens up.[37]"

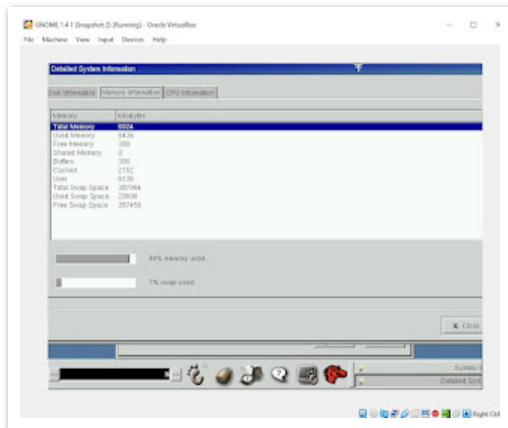
The full quote is:

"I think my feeling was that everything remained in a pilot level, and that it was always important to be able to walk before you tried to run. So, that we were early enough to continue the mapping. And again, the question was when the transition point would come. All of us in the field had been dominated by the development of technology, and everything runs by Sanger's rule; that anytime you get technical development that's two to threefold or more efficient, accurate, cheaper, a whole range of experiments opens up. So, the question was, "Would these be incremental, or completely transformative?" And it turns out that simple changes, like the development of automatic loaders for capillary electro freezes were enough to start bringing the cost down and make things feasible on a large scale. Again, it was Sanger and Washington University, rather than Sanger at that point, who were making decisions for box sequencing. The real question was the point at which one decided that mapping could be dispensed with. And in the historical record, I think that's unclear. We did get to the answer, but it wasn't an obvious choice. The -- how can I put this? The -- one feature of the mapped clones that is -- that remains and has never been completely exploited, is the fact that they reproduce large sections of the NHGRI: Schlessinger, David 2018-03-06 15 03/09/2018 Prepared by National Capitol Contracting 8255 Greensboro Drive, Suite C100 (703) 243-9696 McLean, VA 22102 genome in a form that can be manipulated in a study. So, for example, the A.C. clones we made at the time remain the only source of a complete factor eight gene, a complete factor nine gene. These have both commercial and medical implications. And, they're only available by actually having the clones, which are still achievable in 3D yeast artificial chromosomes at this point. There's the group led by Vladimir Larionov at NCI, which does recombination-based cloning. That's got help with that as well. So, that remains something of great interest."

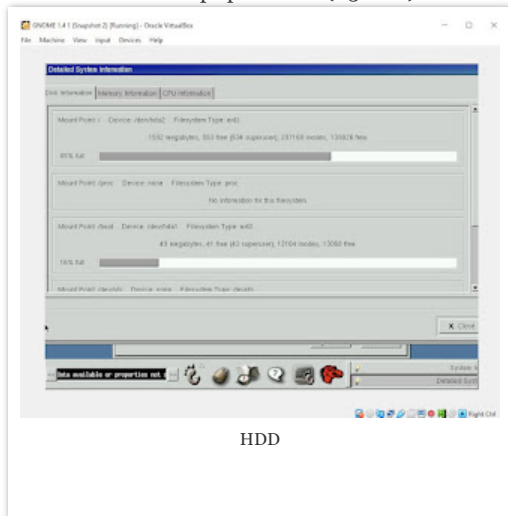
Does AI currently do that? No, not in the way an experienced coder can compile an image. But if you are just a researcher who asks a graduate assistant to find all the books on one topic, something that you could use a search engine yourself, yes, that is something that might save you a little time. So I started to wonder, are any of the AI summaries in Google more comprehensive and specific than something I could find out myself in a Reddit forum that might be the 9th search result on Google? The difference lies in the the speed the AI summary might offer compared to manually reading all the search results myself. In that sense, AI could potentially save me more time in retrieving those answers. For example, when Gemini displayed the results describing how libc5 was a fork of glibc1.2, that might have saved time had I tried to spend a 2nd day compiling linux with libc5 or musl, only to realize I had to use glibc 1.2 and not libc5 or glibc2 (unless I wanted to run AbiWord 3.0 on GTK3, which might use more than 2MB of RAM). **Note:** I am aware RedHat 7.2 uses glibc2, which RedHat has used since **version 4.8** in 1997. What I didn't examine thoroughly is how much extra memory, or source code the early glibc versions require over libc5- if it is negligible for version 2.4, then it might be fine to use GTK3.0. But if it uses lots more RAM- e.g. 4-8MB+additional System RAM=16MB, then applications will require 24-32MB to run, hence an early kernel <2.6) Also one of the results mentioned a compatibility with C89, so there might have been some flags that could have worked only on glibc 1.2 if I didn't want to rely on the newer C99. I don't remember which search result/summary mentioned it, but I saved a **summary** on that too.

Is any of this useful to an experienced coder? Likely not. It is a learning exercise, and I am aware of its limitations. But sometimes I feel like I learn "faster" than hunting and pecking for each item of information. Once a system image can be compiled, a source audit would certainly be used to find any incorrect code and highly redundant components that could have been added by accident. I would hate to call this vibe coding because it requires a semi-informed understanding of what to look for. Perhaps seeing new techniques could be one benchmark for the learner/developer, even if one isn't familiar with them. That places the "tools" in the hands of the researcher, but of course there is a lot to understand from the limits of that capability.

Next tasks: Compare the power consumption of a disk drive boot over a RAMdisk.
Calculate the minimum I/O needed to run a tiny linux OS.



swap space used (~30MB)



HDD

- July 26, 2026 

No comments:

Post a Comment

 Enter Comment

[Home](#)

[Older Post](#)

Subscribe to: [Post Comments \(Atom\)](#)

RedHat 7.2 (2001), Abiword 1.0, & Sanger's Rule towards kernel development

Red Hat 7.2 (2001) with 8MB RAM Red Hat 7.2 System Information Two nights ago (technically Saturday night) I spent some time using free tok...



I'm building a parallel internet, and it's called The Thinnernet

Since 2020, I've taken on a Steve Jobs alter-ego (on a very, very part time basis, and often for humor purposes). The ...



The State of Stateless Linux OSes, and the future of Solar Computing

The title is more of a gag on the other parodied State of the Union addresses, like State of the Onion by Perl Developer Larry Wall: <https://www.perl.com/pertaining/parodies/>



Delivery Optimization for Differential Browser Updates and Desktop Software to Minimize Bandwidth Congestion

I learn a lot about software optimizations by picking the slowest procedure to complete something. Running a software or OS update in th...